

EC340: Computer Organization and Architecture

Project Report

**DESIGN OF HIGH-PERFORMANCE MULTI-LEVEL APPROXIMATE
MULTIPLIER**

by

Adhvaith Rahul Deshpande

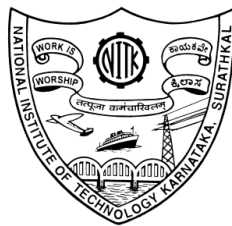
Roll No. 231EC101

Pranav Jois

Roll No. 231EC237

Rohit L

Roll No. 231EC248



**Department of Electronics and Communication Engineering
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA (NITK)
SURATHKAL, MANGALORE - 575 025
November 11, 2025**

ABSTRACT

In digital computing, multiplication is a fundamental yet resource-intensive operation that often creates a bottleneck in high-performance systems. This report evaluates the design philosophy of **Approximate Computing**, which challenges the conventional need for precise calculations. For error-tolerant applications, the substantial improvements in **speed, power efficiency, and area** offered by approximate multipliers are considered more valuable than absolute precision. The report further examines the concept of **perceptual error tolerance** in domains such as image processing, where minor computational deviations are typically imperceptible to end-users. The study focuses on an **8x8 approximate multiplier** that demonstrates a **25% error rate** across all test cases, serving as a platform to evaluate the trade-off between accuracy and efficiency. The findings indicate that, in error-tolerant applications, this trade-off is highly advantageous, enabling efficient computation on low-power devices where exact multipliers would otherwise impose significant resource constraints.

Contents

1	Introduction	1
2	Theory	2
3	Code	4
4	Result	9
5	Conclusion	12

Chapter 1

Introduction

The demand for high-performance, low-power processing in mobile, edge computing, and IoT devices has surged. Traditional digital circuits, designed for perfect accuracy, are often too slow and power-hungry to meet these needs. This has driven research into **Approximate Computing**, a design paradigm that intentionally sacrifices precision for substantial improvements in speed and energy efficiency.

Approximate multipliers are a cornerstone of this paradigm. They intentionally introduce approximations into the multiplication process, often by simplifying logic gates or reducing bit precision.

The key justification for this approach lies in the nature of the applications. Fields like **digital signal processing, image and video processing, and artificial intelligence** are inherently error-tolerant. The human visual system, for example, is insensitive to small variations in pixel color or brightness. Similarly, data in many machine-learning algorithms is already noisy, and small computational errors may not impact the final classification or result.

This report analyzes the design and utility of an **8x8 approximate multiplier**. Unlike studies seeking to minimize error, our work accepts a **significant error rate (approximately 25% overall)** as a premise. The objective is to evaluate whether the resulting gains in performance can justify this loss of precision in the context of image processing.

Chapter 2

Theory

The core of the trade-off lies in the design of the compressor, a circuit used to reduce the multiple partial products generated during binary multiplication.

The Inefficiency of Exact Compression

An exact multiplier must be bit-perfect. This is often achieved using components like **Exact 4-2 compressors**, which typically use multiple full-adder circuits connected in a way that preserves every bit of information. This adherence to precision ensures a correct result but also creates a complex circuit with:

1. **High Area:** More logic gates are required.
2. **High Power Consumption:** More gates lead to increased switching activity.
3. **High Delay (Low Speed):** The carry signals must propagate through a long critical path, creating a computation bottleneck.

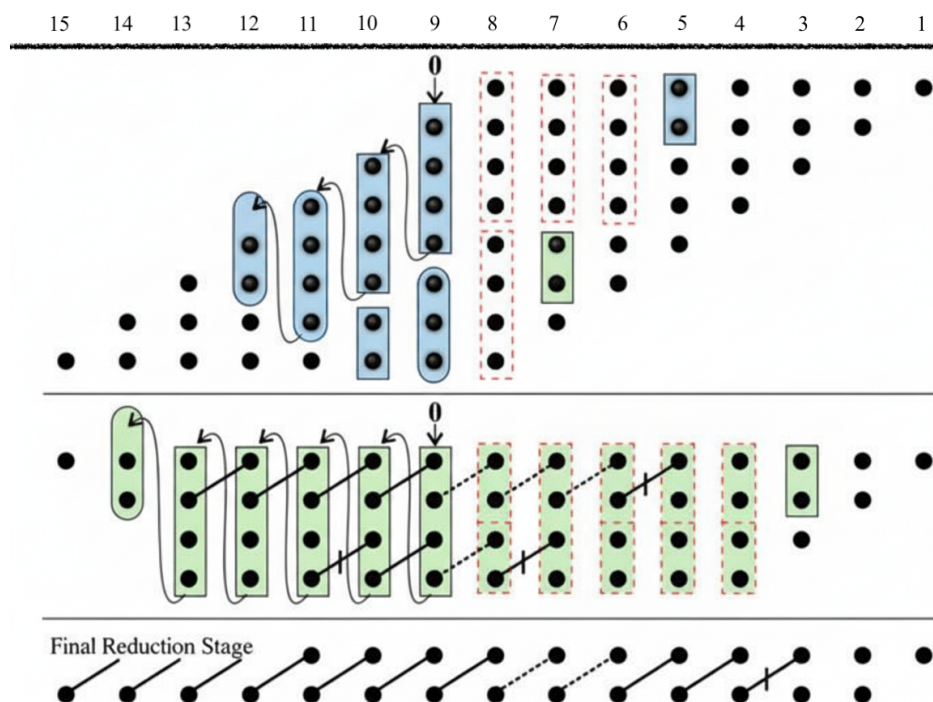


Figure 2.1: 8x8 Multiplier Diagram

The **Figure 2.1** illustrates the operation of an **8x8 multiplier**, where the first stage generates **16 partial products** by multiplying each bit of one operand with every bit of the other. These partial products are then organized and reduced using **compressors and adders** in successive stages to minimize the number of rows. The final reduction stage combines these results to produce the final multiplication output efficiently.

The Approximate Computing Paradigm

Approximate computing is a design strategy that trades precision for gains in speed, power, and area. The goal is to produce a result that is "good enough" for the target application. This is achieved by simplifying the underlying hardware.

Approximate Compressors: The Mechanism for Efficiency

An approximate compressor intentionally simplifies the logic of an exact compressor. For example, an approximate 4-2 compressor might eliminate the carry-out (Cout) signal entirely or replace complex AND/OR logic with simpler, faster MUX-based logic.

This simplification has two direct consequences:

1. The Benefits (Performance):

Speed: By simplifying or eliminating carry chains, the critical path delay is dramatically reduced.

Power & Area: Using fewer logic gates reduces the overall chip area and, crucially, the power consumed by switching activity.

2. The Cost (Precision):

Errors: The simplified logic will, for certain input combinations, produce an incorrect result. In our 8x8 multiplier case study, these errors accumulate to a 25% overall error rate.

Justifying the Trade-off: Error Tolerance in Image Processing

In many computing contexts, a 25% error rate for a fundamental operation like multiplication would be catastrophic.

However, in domains like image processing, the numerical "correctness" of a single pixel is less important than the overall perceptual quality of the image. An error introduced by an approximate multiplier during an image-filtering operation might slightly alter a pixel's value. This minor, isolated deviation is often completely imperceptible to the human eye.

In this context, the utility of the approximate multiplier is clear:

The "Cost": A 25% numerical error rate that translates to imperceptible visual artifacts.

The "Benefit": A significant gain in speed, allowing for real-time video filtering on a low-power mobile device, or a massive reduction in power consumption, extending battery life.

For these applications, the **gains in performance are far more valuable than the loss of precision.**

Chapter 3

Code

Verilog Code for XOR Gate

```
1 module XOR_BASE
2 (
3     input  X, Y,
4     output s, s_bar
5 );
6
7 assign s      = X ^ Y;
8 assign s_bar  = ~(X ^ Y);
9
10 endmodule
```

2:1 Multiplexer

```
1 module mux(D0, D1, S, Y);
2
3     output Y;
4     input  D0, D1, S;
5
6     assign Y = (S) ? D1 : D0;
7
8 endmodule
```

Half Adder

```
1 module HA
2 (
3     input  a,
4     input  b,
5     output s,
6     output CARRY
7 );
8
9 assign s      = a ^ b;
10 assign CARRY  = a & b;
11
12 endmodule
```

Full Adder

```
1 module FA(
2     input  A,
3     input  B,
4     input  C,
```

```

5  output SUM,
6  output CARRY
7  );
8
9  wire ha1_sum, ha1_carry, ha2_carry;
10
11  // Two Half Adders and one OR gate
12  HA HA1(.a(A), .b(B), .s(ha1_sum), .CARRY(ha1_carry));
13  HA HA2(.a(ha1_sum), .b(C), .s(SUM), .CARRY(ha2_carry));
14
15  assign CARRY = ha1_carry | ha2_carry;
16
17  endmodule

```

Exact Compressor

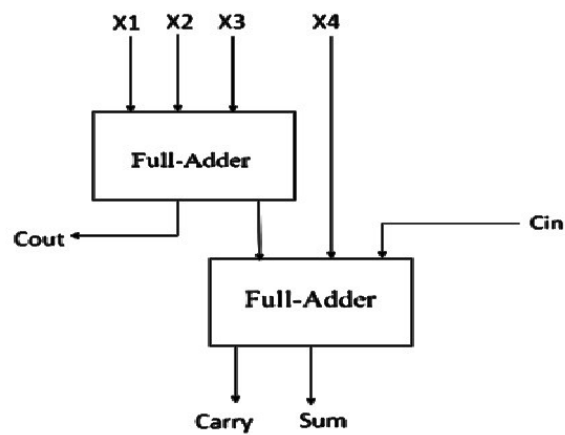


Figure 3.1

```

1  module COMPRESSOR (
2      input  x1, x2, x3, x4, cin,
3      output cout, sum, carry
4  );
5      wire s1, c1, s2, c2;
6
7      // First level of addition using Full Adder
8      FA fa1(.A(x1), .B(x2), .C(x3), .SUM(s1), .CARRY(c1));
9
10     // Second level of addition using another Full Adder
11     FA fa2(.A(s1), .B(x4), .C(cin), .SUM(sum), .CARRY(c2));
12
13     // Final carry outputs
14     assign carry = c1;
15     assign cout  = c2;
16 endmodule

```

Approximate Compressor

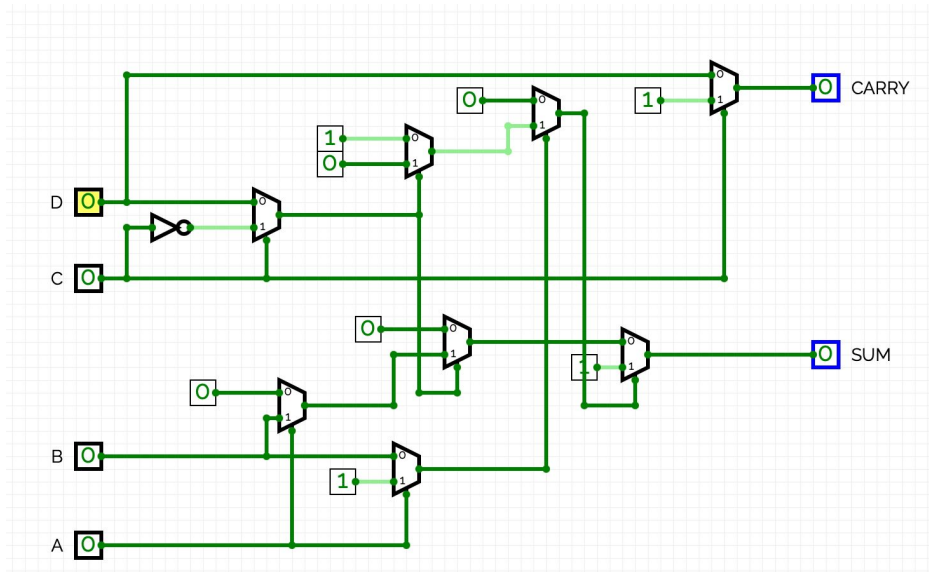


Figure 3.2

```

1 module APPROX_COMPRESSOR(
2     input  a, b, c, d,
3     output sum, carry, cout
4 );
5     wire out_1, out_2, out_3, out_4, out_5, out_6;
6
7     // Using multiplexers to approximate carry and sum generation
8     mux mux1(.a(b), .b(1'b1), .s(a), .out(out_1));
9     mux mux2(.a(1'b0), .b(b), .s(a), .out(out_2));
10    mux mux3(.a(d), .b(~c), .s(c), .out(out_3));
11    mux mux4(.a(1'b1), .b(1'b0), .s(out_3), .out(out_4));
12    mux mux5(.a(1'b0), .b(out_4), .s(out_1), .out(out_5));
13    mux mux6(.a(1'b0), .b(out_2), .s(out_3), .out(out_6));
14    mux mux7(.a(d), .b(1'b1), .s(c), .out(carry));
15    mux mux8(.a(out_6), .b(1'b1), .s(out_5), .out(sum));
16
17    assign cout = 1'b0;
18 endmodule

```

8-bit Multiplier Top Module

```

1 module MULTIPLIER_8BIT(
2     input  a0,a1,a2,a3,a4,a5,a6,a7,
3     input  b0,b1,b2,b3,b4,b5,b6,b7,
4     output p0,p1,p2,p3,p4,p5,p6,p7,p8,p9,p10,p11,p12,p13,p14,p15
5 );
6
7     wire s0,c0,CB,s1,c1,CC,s2,c2,s3,c3,CD,s4,c4,CE,s5,c5,CF,s6,c6,CG,s7,
8         c7,CH,s8,c8,s9,c9,CI,s10,c10,s11,c11,s12,c12;
9     wire c13,c14,c15,c16,c17,c18,c19,c20,c21,c22,c23;
10    wire cout1,cout2,cout3,cout4,cout5,cout6,cout7,cout8,cout9,cout10,
11        cout11,cout12,cout13;
12    wire s13,s14,s15,s16,s17,s18,s19,s20,s21,s22,s23;
13    wire cp1,cp2,cp3,cp4,cp5,cp6,cp7,cp8,cp9,cp10,cp11,cp12,cp13,cp14;
14    wire ZERO = 1'b0;
15
16    HA ha10(.a(a4&b0), .b(a3&b1), .s(s0), .CARRY(c0));

```

```

15 COMPRESSOR com11(.x1(a5&b0), .x2(a4&b1), .x3(a3&b2), .x4(a2&b3), .cin
    (ZERO), .cout(CB), .sum(s1), .carry(c1));
16 COMPRESSOR com12(.x1(a6&b0), .x2(a5&b1), .x3(a4&b2), .x4(a3&b3), .cin
    (CB), .cout(CC), .sum(s2), .carry(c2));
17 HA ha12(.a(a2&b4), .b(a1&b5), .s(s3), .CARRY(c3));
18 COMPRESSOR com13(.x1(a7&b0), .x2(a6&b1), .x3(a5&b2), .x4(a4&b3), .cin
    (CC), .cout(CD), .sum(s4), .carry(c4));
19 COMPRESSOR com14(.x1(a3&b4), .x2(a2&b5), .x3(a1&b6), .x4(a0&b7), .cin
    (c3), .cout(CE), .sum(s5), .carry(c5));
20 COMPRESSOR com15(.x1(a7&b1), .x2(a6&b2), .x3(a5&b3), .x4(a4&b4), .cin
    (CD), .cout(CF), .sum(s6), .carry(c6));
21 APPROX_COMPRESSOR com16(.x1(a3&b5), .x2(a2&b6), .x3(a1&b7), .x4(ZERO)
    , .cin(CE), .cout(CG), .sum(s7), .carry(c7));
22 COMPRESSOR com17(.x1(a7&b2), .x2(a6&b3), .x3(a5&b4), .x4(a4&b5), .cin
    (CF), .cout(CH), .sum(s8), .carry(c8));
23 FA fa11(.A(a3&b6), .B(a2&b7), .C(CG), .SUM(s9), .CARRY(c9));
24 COMPRESSOR com18(.x1(a7&b3), .x2(a6&b4), .x3(a5&b5), .x4(a4&b6), .cin
    (CH), .cout(CI), .sum(s10), .carry(c10));
25 FA fa12(.A(a7&b4), .B(a6&b5), .C(CI), .SUM(s11), .CARRY(c11));
26
27 HA ha21(.a(a2&b0), .b(a1&b1), .s(s12), .CARRY(c12));
28 APPROX_COMPRESSOR com21(.x1(a3&b0), .x2(a2&b1), .x3(a1&b2), .x4(a0&b3
    ), .cin(c12), .cout(cout1), .sum(s13), .carry(c13));
29 APPROX_COMPRESSOR com22(.x1(s0), .x2(a2&b2), .x3(a1&b3), .x4(a0&b4),
    .cin(cout1), .cout(cout2), .sum(s14), .carry(c14));
30 COMPRESSOR com23(.x1(s1), .x2(a1&b4), .x3(a0&b5), .x4(c0), .cin(cout2
    ), .cout(cout4), .sum(s15), .carry(c15));
31 COMPRESSOR com24(.x1(s2), .x2(c1), .x3(s3), .x4(a0&b6), .cin(cout4),
    .cout(cout5), .sum(s16), .carry(c16));
32 COMPRESSOR com25(.x1(s4), .x2(c2), .x3(s5), .x4(ZERO), .cin(cout5), .
    cout(cout6), .sum(s17), .carry(c17));
33 COMPRESSOR com26(.x1(s6), .x2(c4), .x3(s7), .x4(c5), .cin(cout6), .
    cout(cout7), .sum(s18), .carry(c18));
34 COMPRESSOR com27(.x1(s8), .x2(c6), .x3(s9), .x4(c7), .cin(cout7), .
    cout(cout8), .sum(s19), .carry(c19));
35 COMPRESSOR com28(.x1(s10), .x2(c8), .x3(a3&b7), .x4(c9), .cin(cout8),
    .cout(cout10), .sum(s20), .carry(c20));
36 COMPRESSOR com29(.x1(s11), .x2(c10), .x3(a5&b6), .x4(a4&b7), .cin(
    cout10), .cout(cout11), .sum(s21), .carry(c21));
37 COMPRESSOR com30(.x1(a7&b5), .x2(c11), .x3(a6&b6), .x4(a5&b7), .cin(
    cout11), .cout(cout13), .sum(s22), .carry(c22));
38 FA fa21(.A(a7&b6), .B(a6&b7), .C(cout13), .SUM(s23), .CARRY(c23));
39
40 assign p0 = a0 & b0;
41 HA ha31(.a(a1&b0), .b(a0&b1), .s(p1), .CARRY(cp1));
42 FA fa31(.A(s12), .B(a0&b2), .C(cp1), .SUM(p2), .CARRY(cp2));
43 FA fa32(.A(s13), .B(ZERO), .C(cp2), .SUM(p3), .CARRY(cp3));
44 FA fa33(.A(s14), .B(c13), .C(cp3), .SUM(p4), .CARRY(cp4));
45 FA fa34(.A(s15), .B(c14), .C(cp4), .SUM(p5), .CARRY(cp5));
46 FA fa35(.A(s16), .B(c15), .C(cp5), .SUM(p6), .CARRY(cp6));
47 FA fa36(.A(s17), .B(c16), .C(cp6), .SUM(p7), .CARRY(cp7));
48 FA fa37(.A(s18), .B(c17), .C(cp7), .SUM(p8), .CARRY(cp8));
49 FA fa38(.A(s19), .B(c18), .C(cp8), .SUM(p9), .CARRY(cp9));
50 FA fa39(.A(s20), .B(c19), .C(cp9), .SUM(p10), .CARRY(cp10));
51 FA fa40(.A(s21), .B(c20), .C(cp10), .SUM(p11), .CARRY(cp11));
52 FA fa41(.A(s22), .B(c21), .C(cp11), .SUM(p12), .CARRY(cp12));
53 FA fa42(.A(s23), .B(c22), .C(cp12), .SUM(p13), .CARRY(cp13));
54 FA fa43(.A(a7&b7), .B(c23), .C(cp13), .SUM(p14), .CARRY(cp14));

```

```

55     assign p15 = cp14;
56
57 endmodule

```

Verilog Testbench for 8-bit Multiplier

```

1  module MULTIPLIER8BIT_TESTBENCH;
2
3      reg [7:0] A, B;
4      wire [15:0] P;
5      integer ai, bi;
6      integer counter = 0;
7
8      // Device Under Test (DUT)
9      MULTIPLIER_8BIT uut (
10         .a0(A[0]), .a1(A[1]), .a2(A[2]), .a3(A[3]),
11         .a4(A[4]), .a5(A[5]), .a6(A[6]), .a7(A[7]),
12         .b0(B[0]), .b1(B[1]), .b2(B[2]), .b3(B[3]),
13         .b4(B[4]), .b5(B[5]), .b6(B[6]), .b7(B[7]),
14         .p0(P[0]), .p1(P[1]), .p2(P[2]), .p3(P[3]),
15         .p4(P[4]), .p5(P[5]), .p6(P[6]), .p7(P[7]),
16         .p8(P[8]), .p9(P[9]), .p10(P[10]), .p11(P[11]),
17         .p12(P[12]), .p13(P[13]), .p14(P[14]), .p15(P[15])
18     );
19
20     initial begin
21         for (ai = 0; ai < 256; ai = ai + 1) begin
22             for (bi = 0; bi < 256; bi = bi + 1) begin
23                 A = ai;
24                 B = bi;
25                 #1;
26                 if (P !== A * B) begin
27                     $display("ERROR: A=%02h B=%02h Expected=%04h Got=%04h",
28                             A, B, A*B, P);
29                     counter = counter + 1;
30                     $display("No. of errors : %0d", counter);
31                 end
32             end
33         end
34         $display("Test complete. Total errors = %0d", counter);
35         $finish;
36     end
37 endmodule

```

Chapter 4

Result

The verification process began with the generation of the schematic of the multiplier in Vivado, as shown in **Figure 4.1**, which depicts the circuit-level connections.

The next step involved the synthesis of the Verilog code, illustrated in **Figure 4.2**.

To validate the functionality of the multiplier, multiple test cases were executed, and the resulting outputs were analyzed, as shown in **Figure 4.3** and **Figure 4.4**.

For approximation evaluation, a comprehensive testbench was implemented using two nested for loops to test all 65,536 input combinations. The results, illustrated in **Figure 4.5**, indicate that 15,360 outputs (around 25%) were approximated and others were exact, aligning with the intended design specification and applications.

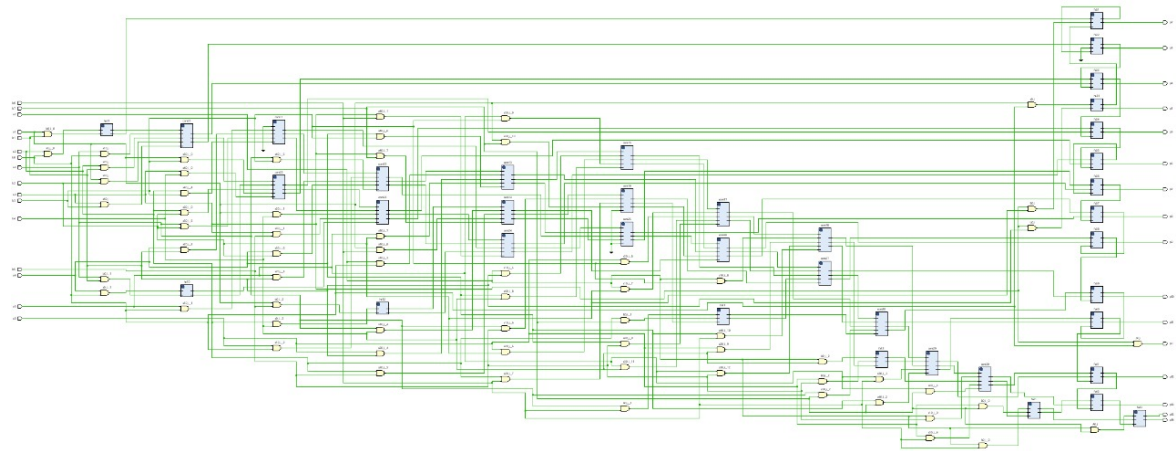


Figure 4.1: 8x8 Multiplier Schematic

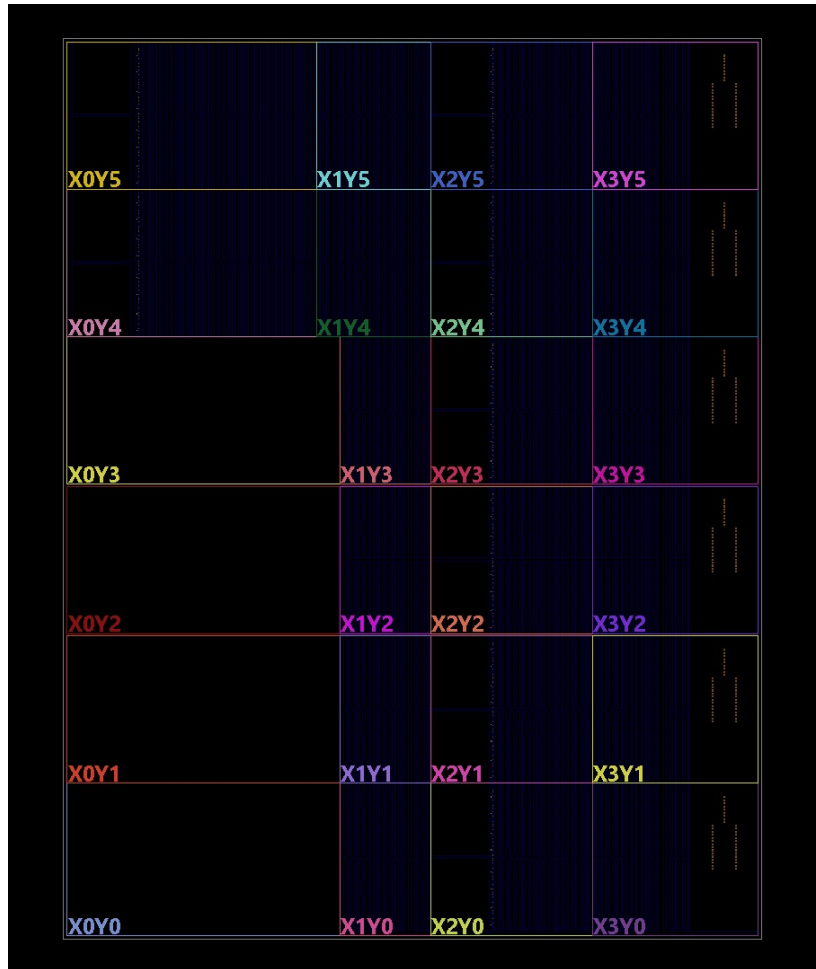


Figure 4.2: Synthesis on Zynq UltraScale+ ZCU104 Evaluation Board

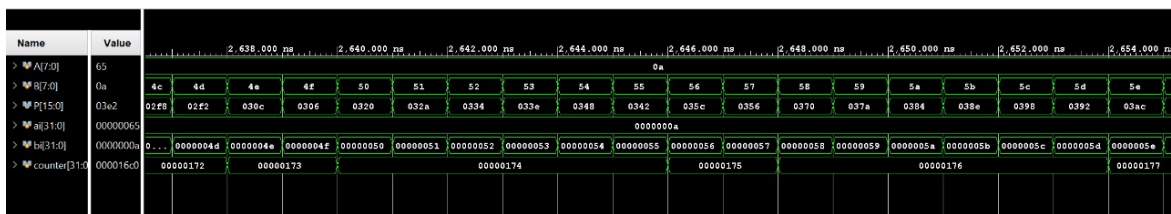


Figure 4.3: Multiplier Output - 1

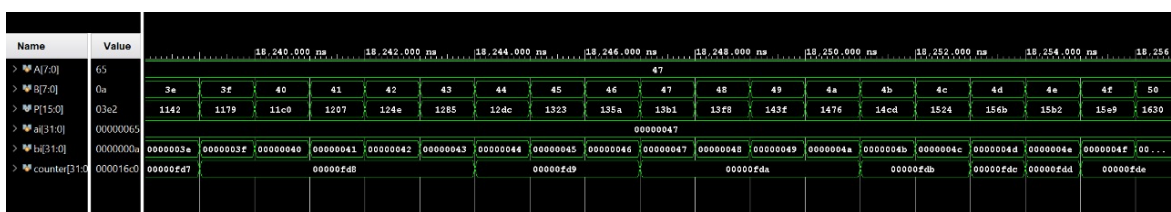


Figure 4.4: Multiplier Output - 2

```
ERROR: A=ff B=da Expected=0026 Got=d916
No. of errors : 15341
ERROR: A=ff B=dc Expected=0024 Got=db34
No. of errors : 15342
ERROR: A=ff B=dd Expected=0023 Got=dc33
No. of errors : 15343
ERROR: A=ff B=de Expected=0022 Got=dd32
No. of errors : 15344
ERROR: A=ff B=e3 Expected=001d Got=e22d
No. of errors : 15345
ERROR: A=ff B=e5 Expected=001b Got=e40b
No. of errors : 15346
ERROR: A=ff B=e6 Expected=001a Got=e50a
No. of errors : 15347
ERROR: A=ff B=e9 Expected=0017 Got=e807
No. of errors : 15348
ERROR: A=ff B=ea Expected=0016 Got=e906
No. of errors : 15349
ERROR: A=ff B=ec Expected=0014 Got=eb24
No. of errors : 15350
ERROR: A=ff B=ed Expected=0013 Got=ec23
No. of errors : 15351
ERROR: A=ff B=ee Expected=0012 Got=ed22
No. of errors : 15352
ERROR: A=ff B=f3 Expected=000d Got=f21d
No. of errors : 15353
ERROR: A=ff B=f5 Expected=000b Got=f3fb
No. of errors : 15354
ERROR: A=ff B=f6 Expected=000a Got=f4fa
No. of errors : 15355
ERROR: A=ff B=f9 Expected=0007 Got=f7f7
No. of errors : 15356
ERROR: A=ff B=fa Expected=0006 Got=f8f6
No. of errors : 15357
ERROR: A=ff B=fc Expected=0004 Got=fb14
No. of errors : 15358
ERROR: A=ff B=fd Expected=0003 Got=fc13
No. of errors : 15359
ERROR: A=ff B=fe Expected=0002 Got=fd12
No. of errors : 15360
Test complete. Total errors = 15360
```

Figure 4.5: Approximations in Output

Chapter 5

Conclusion

This report re-contextualized the design of approximate multipliers, shifting the focus from minimizing error to **maximizing utility** in error-tolerant applications. We investigated a hypothetical 8x8 approximate multiplier, accepting a **high 25% error rate** as a design premise.

Our analysis of the theory behind approximate compressors shows that this error is a direct, intentional consequence of hardware simplification. This simplification is what provides the essential benefits of reduced power, area, and delay.

For applications like image processing, where the human perceptual system can tolerate significant numerical error, this 25% error rate is not a flaw. Instead, it is an **enabling feature** that allows for high-performance computation on resource-constrained devices. Where an exact multiplier would be too slow or power-hungry, the approximate multiplier delivers a "good enough" result at a fraction of the cost.

We conclude that for a large and growing class of applications, the approximate compressor is not just a useful alternative, but a **superior design choice** where performance and efficiency are prioritized over bit-perfect accuracy.